



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD CIENCIAS DE LA COMPUTACIÓN

INGENIERÍA EN SOFTWARE

INFORME 5 GRUPO B

MATERIA:

INGENIERÍA DE REQUERIMIENTOS

INTEGRANTE:

NIEVES SANCHEZ JIMMY SAMUEL CI: 1250907878 - jnievess@uteq.edu.ec

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERÓN

CURSO/PARALELO:

CUARTO SOFTWARE B

QUEVEDO – LOS RÍOS – ECUADOR

PPA 2026 – 2027

Índice

1. Introducción	2
2. Datos generales de la exposición	2
3. Desarrollo de la exposición	2
3.1. Fundamentos conceptuales: CASE, MDE, MDD, MDA y transformaciones M2M y M2T	2
3.2. Técnicas de ingeniería y arquitectura de StarUML	3
3.3. Ventajas y limitaciones de StarUML	3
3.4. Demostración práctica	4
4. Análisis y conclusiones	4

1. Introducción

El presente informe documenta la exposición correspondiente al Grupo B dentro de la asignatura de Ingeniería de Requisitos, en la cual se presentó la herramienta CASE StarUML como recurso de apoyo al modelado UML y a la generación de código a partir de diagramas de clases. La exposición estuvo a cargo de los integrantes Gutiérrez Génesis, Naranjo Anderson y Córdova Mayra, quienes distribuyeron los contenidos abordando los fundamentos conceptuales de las herramientas CASE, las técnicas de ingeniería asociadas al desarrollo dirigido por modelos, y las ventajas y limitaciones de la herramienta, además de una demostración práctica aplicada al proyecto MundiPets. El objetivo del presente documento es reseñar de manera detallada los contenidos expuestos, con el fin de reforzar el proceso de aprendizaje sobre las herramientas orientadas al desarrollo dirigido por modelos.

2. Datos generales de la exposición

En la Tabla 1 se resumen los datos generales correspondientes a la exposición analizada, mientras que en la Tabla 2 se detalla la distribución de los temas abordados por cada integrante.

Grupo	B
Herramienta CASE	StarUML
Integrantes expositores	Gutiérrez Génesis, Naranjo Anderson, Córdova Mayra
Proyecto de referencia	MundiPets, módulo de solicitud de adopción
Asignatura	Ingeniería de Requisitos
Docente	Ing. Guerrero Ulloa Gleiston Cicerón

Integrante	Tema abordado
Gutiérrez, Génesis	Fundamentos de herramientas CASE, paradigmas MDE, MDD y MDA, transformaciones M2M y M2T, aplicación al proyecto MundiPets y demostración práctica
Naranjo, Anderson	Técnicas de ingeniería forward, reverse y round-trip, y arquitectura de StarUML
Córdova, Mayra	Ventajas y limitaciones de StarUML

3. Desarrollo de la exposición

3.1 Fundamentos conceptuales: CASE, MDE, MDD, MDA y transformaciones M2M y M2T

Gutiérrez inició la exposición explicando que las herramientas CASE, correspondientes a las siglas de Computer Aided Software Engineering, son programas diseñados para automatizar y

facilitar el modelado, la generación de código y la documentación del software. A continuación abordó los paradigmas y arquitecturas relacionados con el desarrollo dirigido por modelos: describió al MDE como el enfoque más amplio, en el cual todos los modelos se centran en el proceso de desarrollo del software; presentó al MDD como el enfoque en el que los modelos generan directamente el sistema; y explicó que el MDA, definido por el OMG, separa el diseño conceptual de la lógica de negocio. Adicionalmente describió las transformaciones M2M, correspondientes a la conversión automática entre distintos tipos de modelos, y M2T, correspondientes a la traducción de un diagrama a código fuente.

Finalmente explicó cómo se aplicó StarUML al proyecto de fin de curso MundiPets, específicamente sobre el módulo de solicitud de adopción, empleado como caso de aplicación para la demostración práctica.

3.2 Técnicas de ingeniería y arquitectura de StarUML

Naranjo explicó las técnicas de ingeniería asociadas al desarrollo dirigido por modelos: la técnica forward, correspondiente a la generación de código a partir de un modelo; la técnica reverse, correspondiente a la creación de diagramas a partir de código ya existente; y la técnica round-trip, que permite una sincronización bidireccional entre el modelo y el código, de manera que una modificación realizada en cualquiera de los dos se refleja automáticamente en el otro.

En cuanto a StarUML, señaló que su arquitectura está basada en Node.js y Electron, y que permite el uso de extensiones para adaptarse a distintos lenguajes de programación, lo que la convierte en una herramienta flexible.

3.3 Ventajas y limitaciones de StarUML

Córdova presentó las principales ventajas de StarUML, entre ellas una interfaz ágil e intuitiva, un soporte superior para transformaciones M2T que facilita el seguimiento del proyecto, una alta extensibilidad que permite emplear distintos lenguajes de programación mediante extensiones, y su alineación al estándar UML 2.x. Asimismo expuso las principales limitaciones de la herramienta: la necesidad de una licencia para acceder a las funcionalidades más avanzadas, la obligatoriedad de realizar ajustes manuales por parte del usuario, un consumo considerable de recursos del sistema operativo, la posibilidad de que las extensiones instaladas presenten fallos, y una menor conveniencia para proyectos de gran tamaño. En la Tabla 3 se resumen estos aspectos.

Ventajas	Limitaciones
Interfaz ágil e intuitiva	Requiere licencia para las funcionalidades más avanzadas
Soporte superior para transformaciones M2T	Ajustes manuales obligatorios por parte del usuario
Alta extensibilidad mediante el uso de extensiones	Consumo considerable de recursos del sistema operativo
Alineación al estándar UML 2.x	Las extensiones instaladas pueden presentar fallos

3.4 Demostración práctica

Para la parte demostrativa, Gutiérrez creó un nuevo proyecto en StarUML y, mediante clic derecho sobre la carpeta model, generó un nuevo diagrama de clases. Sobre este diagrama creó la clase csUsuario, a la cual asignó los atributos idUsuario, nombres y apellidos, junto con los métodos Registrar y Autenticar. Posteriormente creó la clase csMascota, con los atributos idMascota, idPropietario y nombre, y los métodos Registrar y actualizarEstado, y la clase csDocumentoVeterinario, con los atributos idDocumento, idMascota e idVeterinario, junto con los métodos Cargar y revisar. Estableció las relaciones correspondientes entre las clases, asignándoles cardinalidades y nombres que facilitarían la comprensión del flujo de trabajo representado, y asignó un estereotipo a cada una de las clases creadas antes de guardar el proyecto.

Con fines de generación de código, Gutiérrez explicó que era necesario dirigirse al menú Tools, acceder al administrador de extensiones e instalar la extensión correspondiente al lenguaje C#. Una vez instalada, mostró que la opción de generación aparece nuevamente en el menú Tools, desde donde seleccionó la carpeta model y la ruta del proyecto correspondiente en Visual Studio 2022, agregando las clases generadas como elementos existentes dentro del proyecto. Señaló que, debido a las limitaciones propias de StarUML, fue necesario realizar ajustes manuales sobre el código generado, entre ellos incorporar las instrucciones de retorno en los métodos de las clases, dado que la herramienta únicamente genera su estructura. Finalmente exportó el código correspondiente a las ocho clases del modelo con sus respectivos ajustes manuales, e instanció la clase csMascota dentro del método Main para demostrar el funcionamiento del código generado.

4. Análisis y conclusiones

A partir de lo expuesto por el Grupo B se puede concluir que StarUML constituye una herramienta CASE flexible y alineada al estándar UML 2.x, cuya arquitectura basada en extensiones le permite adaptarse a distintos lenguajes de programación y escenarios de trabajo. Sin embargo, aspectos como la necesidad de licencia para sus funcionalidades avanzadas, el

consumo considerable de recursos y la obligatoriedad de realizar ajustes manuales sobre el código generado constituyen limitaciones relevantes que deben considerarse antes de adoptarla en proyectos de mayor escala. La demostración realizada sobre el módulo de solicitud de adopción del proyecto MundiPets permitió evidenciar que, si bien la herramienta genera correctamente la estructura de las clases definidas en el modelo, la implementación completa del sistema requiere de un trabajo adicional por parte del desarrollador, lo que confirma su utilidad principalmente como apoyo al proceso de modelado y documentación.